Executions · Rawan

GIU Curriculum and Academic Innovation Management Platform (GCAIMP) · Milestone 2 · executions divided between the team, same split as Procedures.pdf

From the Milestone 2 document (GIU, Winter 2026, submission 15/11/2026, 11:59 pm):

d) **Executions.sql**: an example execution of every stored procedure and function (**EXEC** for procedures, **SELECT** for functions), using data from your insertions.

• The four SQL files must run in this order without errors: Tables.sql, Insertions.sql, Procedures.sql, Executions.sql.

• It is the students' responsibility to make sure that every stored procedure and function executes correctly and returns the expected output.

• Your procedures and functions must handle any errors or exceptions that may happen (e.g., invalid input, records that do not exist, duplicate records, or violated constraints) and show a clear message instead of failing.

How we do it: write your calls in `executions/<YourName>.sql` on your branch, one execution per procedure or function below, in the same order. Use data from Insertions.sql. Add a second call that triggers an error case where it makes sense. Merge into main and the CI glues it into Executions.sql.

Rawan 23 executions (one per procedure / function)

1 As a System Admin, I should be able to:

20. Add a new tool to the catalogue as either an AI Tool or a Teaching Methodology.
    Signature:
    Name: AddTool.
    Type: Stored procedure.
    Input: @Name varchar(150), @Description varchar(1000), @Category varchar(50), @Status varchar(50), @ToolType varchar(30), @Provider varchar(150), @ModelVersion varchar(50), @LicenseType varchar(50), @HostingMode varchar(50), @CostModel varchar(50), @OriginatingFramework varchar(150), @ReferenceSource varchar(150), @RequiredResources varchar(1000).
    Output: Confirmation message or new @ToolID int.

2 As an Academic Instructor, I should be able to:

14. Describe the AI tool integration aspect of an initiative.
    Signature:
    Name: AddAIToolIntegration.
    Type: Stored procedure.
    Input: @InitiativeID int, @IntegrationObjective varchar(1000), @ToolCategory varchar(50), @DataPrivacyNotes varchar(1000), @AcademicIntegrityNotes varchar(1000).
    Output: Confirmation message.

15. Describe the methodology change aspect of an initiative.
    Signature:
    Name: AddMethodologyChange.
    Type: Stored procedure.
    Input: @InitiativeID int, @PedagogicalApproach varchar(150), @DeliveryChange varchar(1000), @ReferenceFramework varchar(150).

Output: Confirmation message.

16. Describe the assessment redesign aspect of an initiative.
    Signature:
    Name: AddAssessmentRedesign.
    Type: Stored procedure.
    Input: @InitiativeID int, @AssessmentTypeAffected varchar(50), @RubricWeightingChanges varchar(1000), @FairnessSafeguards varchar(1000).
    Output: Confirmation message.

17. Link a tool from the catalogue to my initiative.
    Signature:
    Name: AdoptTool.
    Type: Stored procedure.
    Input: @InitiativeID int, @ToolID int.
    Output: Confirmation message.

19. Record a training requirement for an initiative.
    Signature:
    Name: AddTrainingRequirement.
    Type: Stored procedure.
    Input: @InitiativeID int, @ToolID int, @Description varchar(1000), @TargetAudience varchar(150), @RequiredByDate date, @IsMandatory bit.
    Output: Confirmation message.

22. Upload evidence for an initiative, optionally attached to a specific milestone.
    Signature:
    Name: UploadEvidence.
    Type: Stored procedure.
    Input: @InitiativeID int, @MilestoneSeqNo int, @Title varchar(150), @ArtifactType varchar(50), @FilePathOrURL varchar(1000), @Description varchar(1000).
    Output: Confirmation message or new @EvidenceID int.

23. Report the measured impact of an initiative for a semester.
    Signature:
    Name: SubmitOutcomeReport.
    Type: Stored procedure.
    Input: @InitiativeID int, @SemesterID int, @MetricType varchar(50), @QuantitativeValue decimal(10,2), @BaselineValue decimal(10,2), @QualitativeSummary varchar(1000).
    Output: Confirmation message.

24. Record anonymous student feedback about an initiative.
    Signature:
    Name: RecordStudentFeedback.
    Type: Stored procedure.
    Input: @InitiativeID int, @SemesterID int, @Rating int, @Comment varchar(1000), @CategorySentiment varchar(50), @CollectionMethod varchar(50).
    Output: Confirmation message.

3 As a Teaching Assistant, I should be able to:

7. View all evidence uploaded for an initiative I am assigned to.
    Signature:

Name: ViewInitiativeEvidence.
Type: Function.
Input: @InitiativeID int.
Output: Table of evidence with title, artifact type, milestone and upload date.

5 As a Dean, I should be able to:

12. Schedule a training session (online or in person) for a tool.
    Signature:
    Name: ScheduleTrainingSession.
    Type: Stored procedure.
    Input: @ToolID int, @Title varchar(150), @Topic varchar(150), @Description varchar(1000), @SessionDate date, @DurationMinutes int, @Trainer varchar(150), @Capacity int, @Mode varchar(20), @Platform varchar(50), @MeetingLink varchar(1000), @RecordingAvailable bit, @Venue varchar(150), @Campus varchar(50), @RequiredEquipment varchar(1000).
    Output: Confirmation message or new @SessionID int.
13. View the attendance list of a training session.
    Signature:
    Name: ViewTrainingAttendance.
    Type: Function.
    Input: @SessionID int.
    Output: Table of attending members with name, rank and email.
14. Check training compliance for the initiatives in my faculty.
    Signature:
    Name: GetFacultyTrainingCompliance.
    Type: Function.
    Input: @FacultyID int.
    Output: Table showing initiative, training requirement, number of assigned members and number who attended matching training.
15. View the measured impact of an initiative compared to its baseline.
    Signature:
    Name: GetInitiativeImpact.
    Type: Function.
    Input: @InitiativeID int.
    Output: Table showing semester, metric type, baseline, value and percentage change.
16. Calculate the average student feedback rating of an initiative in a semester.
    Signature:
    Name: GetAverageFeedbackRating.
    Type: Function.
    Input: @InitiativeID int, @SemesterID int.
    Output: Average rating decimal(3,2).

6 As an University Management member, I should be able to:

5. View how widely each tool has been adopted.
    Signature:
    Name: GetToolAdoptionReport.
    Type: Function.
    Input: Nothing.

Output: Table showing tool, number of initiatives, number of faculties and number of trained members.

10. Change the status of a tool university-wide (e.g., Approved, Under Review, Suspended).
    Signature:
    Name: UpdateToolStatus.
    Type: Stored procedure.
    Input: @ToolID int, @Status varchar(50).
    Output: Confirmation message.

7 As an Academic Member, I should be able to:

6. View upcoming training sessions for a tool.
   Signature:
   Name: ViewAvailableTrainingSessions.
   Type: Function.
   Input: @ToolID int.
   Output: Table of sessions with date, mode, trainer and remaining seats.

7. Check the number of remaining seats in a training session.
   Signature:
   Name: GetRemainingSeats.
   Type: Function.
   Input: @SessionID int.
   Output: Number of remaining seats int.

8. Register for a training session (only if seats are available).
   Signature:
   Name: RegisterForTraining.
   Type: Stored procedure.
   Input: @SessionID int, @MemberID int.
   Output: Confirmation message.

9. Cancel my registration in a training session before it takes place.
   Signature:
   Name: CancelTrainingRegistration.
   Type: Stored procedure.
   Input: @SessionID int, @MemberID int.
   Output: Confirmation message.

10. View my training history.
    Signature:
    Name: ViewMyTrainingHistory.
    Type: Function.
    Input: @MemberID int.
    Output: Table of attended sessions with tool, date and mode.

11. View the mandatory training I still need for the initiatives I am assigned to.
    Signature:
    Name: ViewMyPendingTraining.
    Type: Function.
    Input: @MemberID int.
    Output: Table showing initiative, requirement, tool and required-by date.